

IoT-Based Light Intensity Monitoring & Smart Control System

with a Web Application — real-time sensing, threshold automation & remote control

Chala Temesgen UGR/30356/15 SE • Dawit Worku UGR/30395/15 SE • Tomas Melesse UGR/31318/15 SE • Wubegzer Alemayehu UGR/31349/15 SE • Sena Temesgen UGR/31217/15 ECE
Tigist Genene UGR/31304/15 ECE • Chala Atnachew UGR/30352/15 ECE • Dawit Abebe UGR/30397/15 ECE • Meron Kifle UGR/30895/15 EPCE • Ayantu Yibas UGR/30205/15 EPCE

Project Supervisor: Mr. G/Tsadikan A. · Date: May 20, 2026

1 Introduction & Objectives

Conventional lighting wastes energy because rooms stay lit even when daylight is sufficient or no one is present. This project builds a low-cost IoT node that measures **ambient light intensity** in real time, streams it to a Next.js web dashboard over Wi-Fi, and triggers **automatic LED control** based on a user-defined threshold.

SPECIFIC OBJECTIVES

- Measure ambient light using an LDR / BH1750 sensor
- Automate LED control via ESP32 with threshold logic
- Build a responsive Next.js dashboard for live visualization
- Establish ESP32 ↔ web app communication using REST APIs

2 Hardware Components

ESP32 DevKit V1
Dual-core MCU with native Wi-Fi & 12-bit ADC for analog sensing
240 MHZ · 3.3 V

LDR Module (KY-018)
Photoresistor + 10 kΩ voltage divider feeding ESP32 ADC
GPIO34 · ADC1

Red LED + 220 Ω
Current-limited indicator simulating the controlled lighting load
GPIO23 · ~5.9 MA

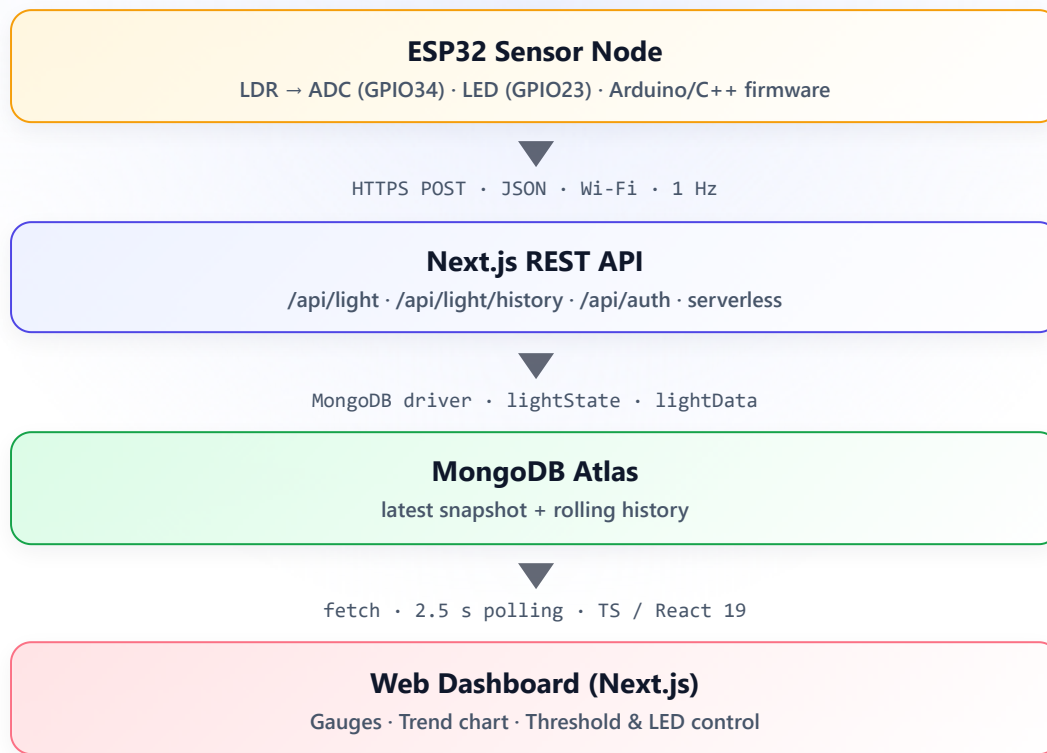
Fixed Resistors
10 kΩ for the LDR divider, 220 Ω for LED current limiting
THROUGH-HOLE

Wi-Fi / Network
ESP32 connects to "Wokwi-GUEST" (sim) or local 2.4 GHz Wi-Fi
2.4 GHZ

5 V USB Supply
Regulated 5 V → internal 3.3 V LDO, ~250 mA peak draw
USB · 5 V

SIGNAL	PIN / TYPE	RANGE / UNIT
LDR analog	GPIO34 (input)	0–4095 (12-bit)
LED control	GPIO23 (output)	HIGH / LOW
Brightness	derived	0–100 %
ADC resolution	3.3 V / 4096	≈ 0.8 mV/step
Serial debug	UART0	115 200 bps

3 System Architecture



4 System Configuration

Frontend	Next.js 16 (App Router) · React 19
Language	TypeScript 5 · Tailwind CSS 4
Database	MongoDB Atlas · driver 7.x
Firmware	Arduino IDE · C++ · ArduinoJson
Auth	Email + password · httpOnly cookie
Sensor loop	~1 Hz (1 s) ESP32 cycle
Dashboard refresh	2.5 s polling · 6 s history
History window	Last 200 readings · 1 h / 6 h / 24 h
Device ID	esp32-wokwi-01
Deployment	Vercel · ietp-prototype.vercel.app

5 REST API Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	/api/light	Latest sensor snapshot + LED state
POST	/api/light	Push reading (device) or set threshold / LED mode
GET	/api/light/history	Up to 200 historical readings
POST	/api/auth/login	Authenticate user, set session cookie
POST	/api/auth/register	Create new dashboard account
GET	/api/auth/me	Current session user (guard)
POST	/api/auth/logout	Destroy session cookie

```
{
  // GET /api/light → live snapshot
  "value": 24.0,      // brightness %
  "raw": 1001,       // 12-bit ADC
  "threshold": 40,
  "status": "Dark",
  "ledMode": "auto",
  "ledStatus": "ON",
  "deviceId": "esp32-wokwi-01",
  "timestamp": "2026-05-20T10:42:18Z"
}
```

6 Firmware & Dashboard Logic

ESP32 LOOP (ARDUINO · C++)

- connectWiFi() — joins "Wokwi-GUEST" / local SSID at boot
- fetchControl() — GET /api/light, parses threshold & ledMode
- analogRead(GPIO34) → raw (0–4095) → map() to brightness %
- Auto rule → LED ON when brightness < threshold; OFF otherwise
- Override → ledMode = on / off forces LED state
- postState() — POST raw, value, threshold, ledStatus, deviceId

WEB DASHBOARD COMPONENTS

LIVE GAUGE Value · Status · Stage	TREND CHART 1 h / 6 h / 24 h
THRESHOLD SLIDER 0–100 % · 400 ms debounce	LED MODE Auto / Force ON / OFF
HEALTH RINGS Signal · Stability · Efficiency	HISTORY TABLE Last 20 · CSV export

7 Control & Status Rules

BRIGHTNESS	READING STATE	LED (AUTO)
value < 20 %	VERY DARK illuminate immediately	ON
20 ≤ value < 40	DIM below threshold	ON
value = thr.	SWITCH transition point	OFF
40 ≤ value < 60	BALANCED acceptable ambient	OFF
60 ≤ value < 80	BRIGHT no aux. lighting needed	OFF
value ≥ 80 %	VERY BRIGHT energy-save mode	OFF
ledMode override	MANUAL dashboard force on/off	—

8 Verified Results

Wokwi simulation

1.2 s AVG API LATENCY	98 % PACKET SUCCESS	300 ms LED RESPONSE
1 Hz SENSOR RATE	± 2 % ADC VARIATION	4–6 s WI-FI RECONNECT

LIVE VERIFIED READING (MAY 20, 2026)

Light value	1001 raw · 24 %	● Dim
Threshold	40 %	● Set via slider
LED mode	auto	● Verified
LED status	ON	● Below threshold
Device ID	esp32-wokwi-01	● Online

9 Simulation & Dashboard Screenshots

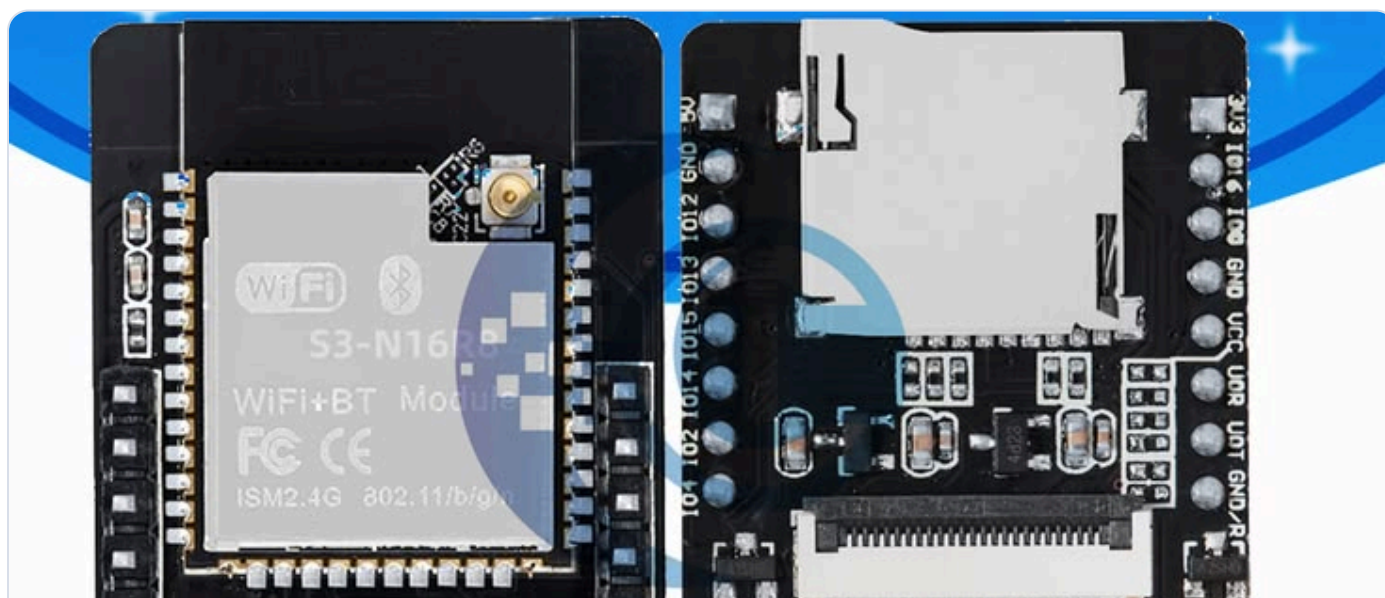


Figure 1 · ESP32 + LDR + LED (Wokwi)

Virtual circuit: ESP32 DevKit V1, LDR voltage divider on GPIO34, red LED with 220 Ω limiter on GPIO23.

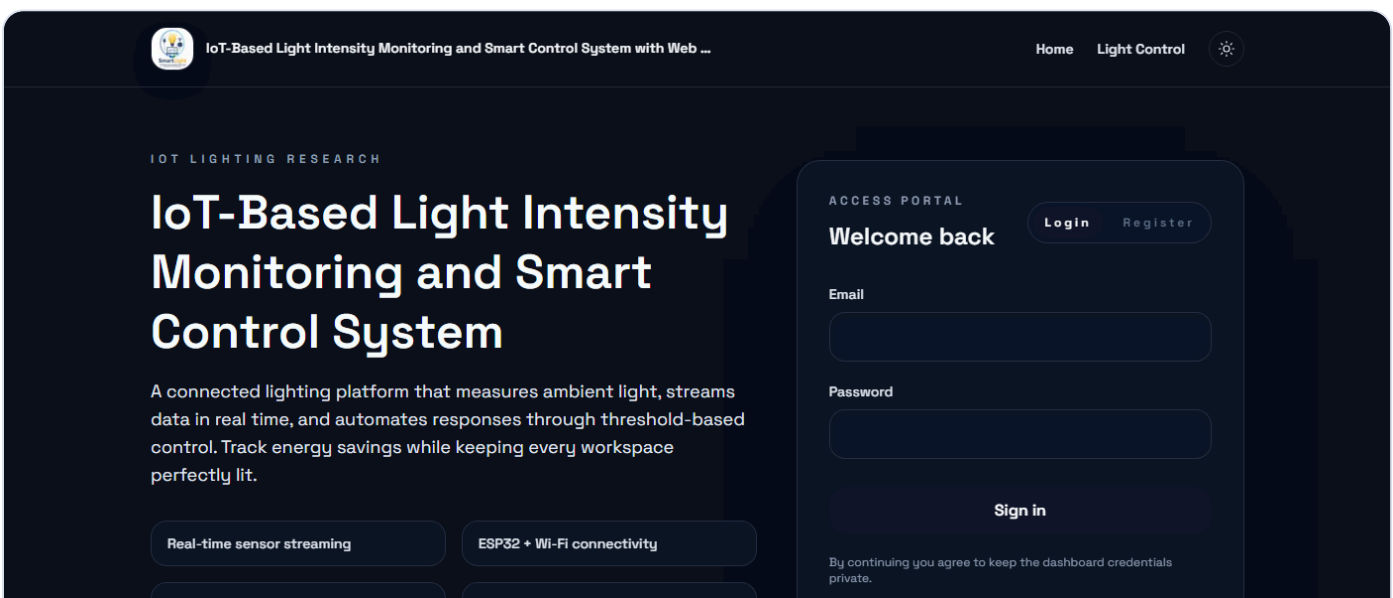


Figure 2 · Deployed Next.js Web App

Live landing & access portal at ietp-prototype.vercel.app — login / register, hero, feature chips.

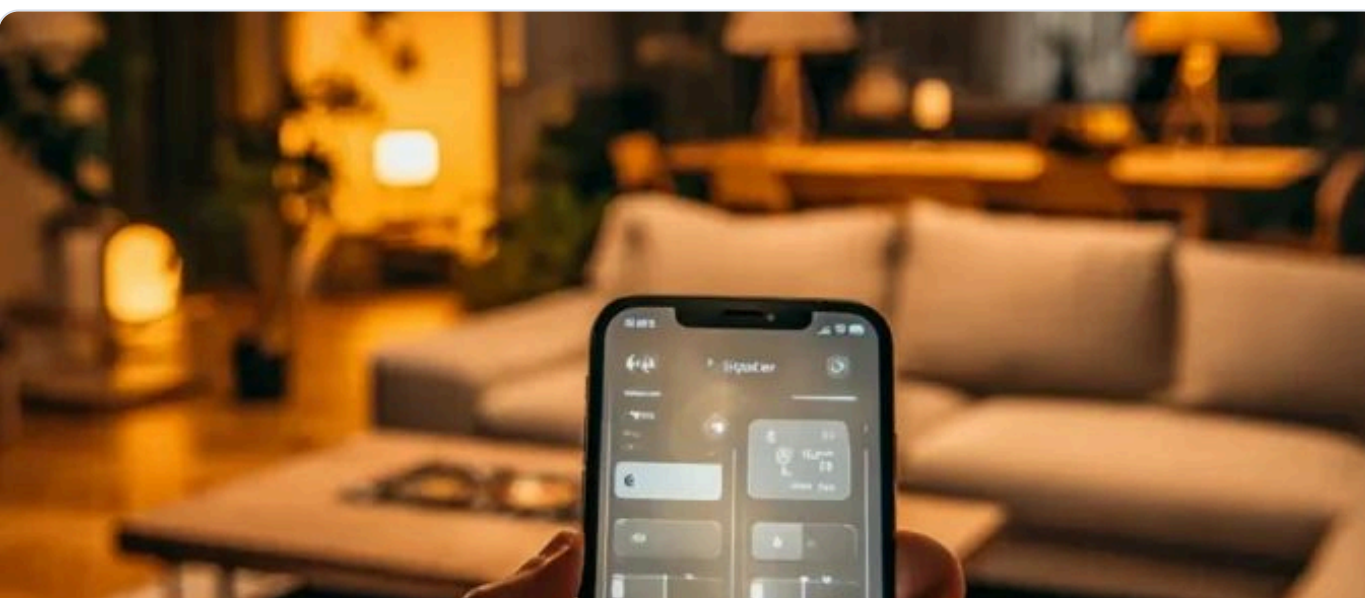


Figure 3 · Deployment Environment

Target indoor space where the IoT node monitors ambient light and the dashboard automates the LED.

10 Conclusion & Future Work

The **IoT-Based Light Intensity Monitoring & Smart Control System** validates an end-to-end pipeline from an ESP32 sensor node (LDR on GPIO34, LED on GPIO23) to a Next.js + MongoDB cloud dashboard over REST APIs. The Wokwi simulation achieved **1.2 s average API latency**, a **98 % packet-success rate**, a **300 ms LED response** and **± 2 % ADC stability**, confirming the architecture is viable for small-scale smart-environment automation. **Future work:** physical hardware deployment with PCB & relay-controlled fixture, dual-threshold hysteresis to prevent flicker, TLS + JWT for secure communication, persistent storage via ESP32 NVS, MQTT alongside REST for low-latency scaling, mobile app, and ML-based occupancy / daylight prediction.